T3000 API & Protocol

Outputs · built Nov 13, 2026

Outputs

Control analog and digital output points for actuators, dampers, valves, and equipment.

Overview

Output points represent physical control signals sent to actuators, variable frequency drives (VFDs), dampers, valves, and other controlled devices. Outputs can operate in automatic mode (controlled by programs) or manual override mode.

Output Types

Analog Outputs (AO)

Continuous control signals:

Common Applications:

- **Damper Control**: Modulating dampers (0-100%)
- **Valve Control**: Hot/cold water valves
- **VFD Speed Control**: Fan/pump speed (0-100%)
- **Setpoint Signals**: Temperature/pressure setpoints
- **4-20mA Signals**: Pneumatic transducers
- **0-10V Signals**: Electronic actuators

Properties:

- **Command Value**: Desired output (0-100%)
- **Feedback Value**: Actual position reading
- **Units**: % (percent), or engineering units
- **Range**: Min/max output limits
- **Override**: Manual control capability

Binary Outputs (BO)

On/off control signals:

Common Applications:

- **Equipment Start/Stop**: Fans, pumps, heaters
- **Relay Control**: Contactors, solenoids
- **Enable/Disable**: System enable signals
- **Alarm Outputs**: Strobe lights, horns
- **Safety Shutoffs**: Emergency stops

States:

- **Active** (1, On, Energized)
- **Inactive** (0, Off, De-energized)

Outputs Page Interface

Table View

#	Label	Description	Command	Feedback	Mode	Override	Status	Update
1	Damper_Pos	Supply Damper	45%	44%	Auto	No	✅ Normal	2s ago
2	Fan_Speed	Supply Fan VFD	75%	75%	Auto	No	✅ Normal	2s ago
3	HW_Valve	Hot Water Valve	0%	0%	Off	Yes	🔧 Override	2s ago
4	Pump_Run	Chilled Water Pump	On	On	Auto	No	✅ Normal	2s ago

Features

Real-Time Control:

- View current command values
- See actual feedback positions
- Monitor mode status (Auto/Manual/Off)
- Identify override conditions

Manual Override:

- Click output row to override
- Set manual value
- Return to automatic mode

- Emergency stop capability

Batch Operations:

- Select multiple outputs
- Batch mode changes
- Release all overrides
- Export configuration

Output Control

Operating Modes

Automatic Mode:

- Output controlled by programs
- Normal operating condition
- Responds to control algorithms
- Cannot be manually adjusted

Manual Mode:

- User sets output value directly
- Program control disabled
- Value held until released
- Indicated with 🔧 icon

Off Mode:

- Output disabled
- Fixed at 0% or Off state
- Safety/maintenance condition
- Must be manually enabled

Manual Override

To override an output:

1. Click on output row
2. Select **Manual Override** tab
3. Set desired value (0-100% or On/Off)
4. Click **Apply Override**
5. Output shows 🔧 override indicator

Release Override:

1. Click overridden output
2. Click **Release Override** button
3. Output returns to automatic mode

⚠ **Warning**: Manual overrides persist until explicitly released. Remember to return outputs to Auto mode after maintenance.

Output Configuration

Basic Settings

Identification:

- **Label**: Output name (e.g., "Supply_Damper")
- **Description**: Detailed description
- **Control Type**: Modulating/Two-Position
- **Panel**: Parent control panel

Control Parameters:

- **Range**: Output range (0-100%, 0-10V, 4-20mA)
- **Default**: Default value when disabled
- **Priority**: Control priority level
- **Relinquish**: Auto-release timeout

Advanced Settings

PID Tuning (if applicable):

```
TEXT

Proportional (P):  2.0
Integral (I):      0.5
Derivative (D):    0.1
Setpoint:          72°F
Deadband:          1°F
```

Limits:

- **High Limit**: Maximum allowed output
- **Low Limit**: Minimum allowed output
- **Rate Limit**: Maximum change per second

- **Deadband**: Minimum change threshold

Safety:

- **Failsafe Value**: Value on communication loss
- **Timeout**: Watchdog timer (seconds)
- **Lockout**: Disable manual override
- **Priority Array**: BACnet priority levels

Feedback Monitoring

Position Feedback

Many outputs have feedback sensors:

Feedback Types:

- **Position Sensor**: Actual damper/valve position
- **Speed Sensor**: VFD actual speed
- **Flow Switch**: Proof of flow
- **Status Contact**: Relay feedback

Feedback Comparison:

```
TEXT

Command:    75%
Feedback:   73%
Error:      2%
Status:     ✅ Normal (within tolerance)
```

Feedback Errors:

- Command-feedback difference > tolerance
- Indicates mechanical issue
- Generates alarm
- May indicate:
  - Stuck actuator
  - Broken linkage
  - Sensor failure
  - Insufficient power

Output Sequences

Start/Stop Sequences

Equipment Start Sequence:

1. Pre-start checks
2. Enable output
3. Ramp to setpoint
4. Verify feedback
5. Confirm running status

Equipment Stop Sequence:

1. Ramp down gradually
2. Allow time delay
3. Disable output
4. Verify off status
5. Log stop event

Interlocks

Safety Interlocks:

- Prevent unsafe conditions
- Require prerequisite states
- Automatic shutdown on fault
- Override prevention

Example: VFD Interlock

```
TEXT
```

```
IF (Filter_Status == Dirty) THEN
    VFD_Enable = OFF
    Alarm_Active = TRUE
END IF
```

Trending

Output Trends

Monitor output behavior:

Trend Charts:

- Command vs. Feedback comparison
- Mode changes over time
- Override history
- Performance analysis

Useful Metrics:

- Average position
- Cycling frequency
- Override duration
- Error magnitude

Alarms

Output Alarms

Common alarm conditions:

Feedback Error:

```
TEXT

Condition:   |Command - Feedback| > 10%
Duration:    > 60 seconds
Priority:    Medium
Action:      Notify, log event
```

Override Active:

```
TEXT

Condition:   Output in manual mode
Duration:    > 2 hours
Priority:    Low
Action:      Remind to release
```

Safety Limit:

```
Condition:   Output at high/low limit
Duration:    > 5 minutes
Priority:    High
Action:      Alert maintenance
```

Best Practices

Naming Conventions

Use consistent output naming:

```
Format: [Equipment]_[Function]_[Type]

Examples:
- AHU1_Supply_Damper
- Boiler1_HW_Valve
- ExhFan_Speed_VFD
- Pump2_Run_Command
```

Control Strategy

1. **Gradual Changes**
   - Use rate limiting
   - Avoid abrupt changes
   - Prevent equipment stress
2. **Feedback Verification**
   - Monitor feedback vs. command
   - Set appropriate tolerances
   - Investigate persistent errors
3. **Override Management**
   - Document override reason
   - Set time limit reminders
   - Review daily overrides
   - Release after maintenance

Safety Considerations

1. **Emergency Override**
    - Know how to quickly disable outputs
    - Test emergency stop procedures
    - Document shutdown sequences
2. **Failsafe Settings**
    - Configure safe default values
    - Set appropriate timeouts
    - Test communication loss behavior
3. **Authorization**
    - Limit override permissions
    - Log all manual interventions
    - Review override history

Troubleshooting

Output Not Responding

Symptoms: Command changes but no response

Solutions:

1. Check device connection
2. Verify not in manual override
3. Test output in manual mode
4. Check wiring and power
5. Verify output enable status

Feedback Mismatch

Symptoms: Command ≠ Feedback

Solutions:

1. Check feedback sensor wiring
2. Calibrate position feedback
3. Verify actuator is working
4. Check for mechanical binding

5. Test actuator manually

Unstable Control

Symptoms: Output hunting/oscillating

Solutions:

1. Tune PID parameters
2. Increase deadband
3. Add rate limiting
4. Check sensor noise
5. Verify loop stability

Next Steps

- Variables - Calculated variables
- Programs - Control programs
- PID Loops - PID control
- Schedules - Time-based control

Outputs

BACnet Output Control

Writing Output Values

TYPESCRIPT

```typescript
// Write single output via REST API
await fetch(`http://localhost:9103/api/devices/389001/points/OUT1`, {
  method: 'PUT',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    value: 75.0,
    priority: 8  // Manual operator (BACnet priority)
  })
});

// BACnet write property
import { BACnetClient } from '@temco/t3000-sdk';

const client = new BACnetClient({ port: 47808 });

await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'analog-output',
  objectInstance: 1,
  property: 'present-value',
  value: 75.0,
  priority: 8
});
```

BACnet Priority Array

TYPESCRIPT

```typescript
interface PriorityLevel {
  priority: number;
  value: number | null;
  description: string;
}

const priorityArray: PriorityLevel[] = [
  { priority: 1, value: null, description: 'Manual Life Safety' },
  { priority: 2, value: null, description: 'Automatic Life Safety' },
  { priority: 3, value: null, description: 'Available' },
  { priority: 4, value: null, description: 'Available' },
  { priority: 5, value: null, description: 'Critical Equipment' },
  { priority: 6, value: null, description: 'Minimum On/Off' },
  { priority: 7, value: null, description: 'Available' },
  { priority: 8, value: 75.0, description: 'Manual Operator' },
  { priority: 9, value: null, description: 'Available' },
  { priority: 10, value: 72.0, description: 'Auto Control' },
  { priority: 11, value: null, description: 'Available' },
  { priority: 12, value: null, description: 'Available' },
  { priority: 13, value: null, description: 'Available' },
  { priority: 14, value: null, description: 'Available' },
  { priority: 15, value: null, description: 'Available' },
  { priority: 16, value: 50.0, description: 'Default/Relinquish' }
];

// Effective value is the highest priority non-null value
function getEffectiveValue(priorityArray: PriorityLevel[]): number {
  for (const level of priorityArray) {
    if (level.value !== null) {
      return level.value;
    }
  }
  return 0; // Failsafe if all null
}
```

Relinquish Override

TYPESCRIPT

```typescript
// Release manual override (write null at priority 8)
await client.writeProperty({
  deviceInstance: 389001,
  objectType: 'analog-output',
  objectInstance: 1,
  property: 'present-value',
  value: null,  // null relinquishes this priority
  priority: 8
});
```

Output Control Algorithms

PID Output Calculation

TYPESCRIPT

```typescript
class PIDController {
  private integral = 0;
  private lastError = 0;
  private lastTime = Date.now();

  constructor(
    private kp: number,
    private ki: number,
    private kd: number,
    private outputMin: number = 0,
    private outputMax: number = 100
  ) {}

  calculate(setpoint: number, processValue: number): number {
    const now = Date.now();
    const dt = (now - this.lastTime) / 1000; // seconds

    const error = setpoint - processValue;

    // Proportional term
    const p = this.kp * error;

    // Integral term with anti-windup
    this.integral += error * dt;
    const i = this.ki * this.integral;

    // Derivative term
    const derivative = (error - this.lastError) / dt;
    const d = this.kd * derivative;

    // Calculate output
    let output = p + i + d;

    // Apply limits and anti-windup
    if (output > this.outputMax) {
      output = this.outputMax;
```

```
      this.integral -= error * dt; // Prevent windup
    } else if (output < this.outputMin) {
      output = this.outputMin;
      this.integral -= error * dt;
    }

    this.lastError = error;
    this.lastTime = now;

    return output;
  }

  reset() {
    this.integral = 0;
    this.lastError = 0;
  }
}
```

Rate Limiting

TYPESCRIPT

```typescript
class RateLimiter {
  private lastValue: number;
  private lastTime: number;

  constructor(
    initialValue: number,
    private maxRate: number  // Max change per second
  ) {
    this.lastValue = initialValue;
    this.lastTime = Date.now();
  }

  limit(targetValue: number): number {
    const now = Date.now();
    const dt = (now - this.lastTime) / 1000;

    const maxChange = this.maxRate * dt;
    const requestedChange = targetValue - this.lastValue;

    let actualChange = requestedChange;
    if (Math.abs(requestedChange) > maxChange) {
      actualChange = maxChange * Math.sign(requestedChange);
    }

    this.lastValue += actualChange;
    this.lastTime = now;

    return this.lastValue;
  }
}

// Example: Limit damper to 10% per second
const limiter = new RateLimiter(0, 10);

const command = 75;
const limited = limiter.limit(command);  // Gradually ramps to target
```

Deadband Control

TYPESCRIPT

```typescript
function applyDeadband(
  currentOutput: number,
  targetOutput: number,
  deadband: number
): number {
  const difference = Math.abs(targetOutput - currentOutput);

  if (difference < deadband) {
    return currentOutput;  // No change within deadband
  }

  return targetOutput;
}

// Example: 2% deadband on damper control
const current = 45.0;
const target = 46.5;
const output = applyDeadband(current, target, 2.0);  // Returns 46.5
```

Feedback Monitoring

Command-Feedback Comparison

TYPESCRIPT

```typescript
interface FeedbackStatus {
  command: number;
  feedback: number;
  error: number;
  errorPercent: number;
  status: 'normal' | 'warning' | 'fault';
}

function checkFeedback(
  command: number,
  feedback: number,
  tolerance: number = 5.0  // Percent
): FeedbackStatus {
  const error = Math.abs(command - feedback);
  const errorPercent = (error / command) * 100;

  let status: 'normal' | 'warning' | 'fault' = 'normal';
  if (errorPercent > tolerance * 2) {
    status = 'fault';
  } else if (errorPercent > tolerance) {
    status = 'warning';
  }

  return {
    command,
    feedback,
    error,
    errorPercent,
    status
  };
}

// Example: Check VFD feedback
const status = checkFeedback(75.0, 73.0, 5.0);
console.log(`Error: ${status.errorPercent.toFixed(1)}%, Status: ${status.status}`);
```

Stuck Output Detection

TYPESCRIPT

```typescript
class StuckOutputDetector {
  private history: number[] = [];
  private maxSamples = 20;

  update(feedback: number): boolean {
    this.history.push(feedback);

    if (this.history.length > this.maxSamples) {
      this.history.shift();
    }

    if (this.history.length < this.maxSamples) {
      return false;  // Not enough data
    }

    // Check if all values are identical
    const first = this.history[0];
    const allSame = this.history.every(v => Math.abs(v - first) < 0.1);

    return allSame;  // True if stuck
  }
}
```

Output Sequencing

Equipment Start Sequence

TYPESCRIPT

```typescript
async function startEquipment(outputId: string): Promise<boolean> {
  try {
    // 1. Pre-start checks
    const safeToStart = await performSafetyChecks(outputId);
    if (!safeToStart) {
      console.error('Safety checks failed');
      return false;
    }

    // 2. Enable output
    await writeOutput(outputId, 100, 8);

    // 3. Wait for proof of operation
    await sleep(5000);

    // 4. Verify feedback
    const feedback = await readFeedback(outputId);
    if (feedback < 90) {
      console.error('Failed to start - no feedback');
      await writeOutput(outputId, 0, 8);
      return false;
    }

    // 5. Log start event
    await logEvent('equipment-start', { outputId, timestamp: Date.now() });

    return true;
  } catch (error) {
    console.error('Start sequence failed:', error);
    return false;
  }
}
```

Interlock Logic

TYPESCRIPT

```typescript
class InterlockManager {
  private interlocks = new Map<string, () => boolean>();

  addInterlock(outputId: string, condition: () => boolean) {
    this.interlocks.set(outputId, condition);
  }

  checkInterlocks(outputId: string): boolean {
    const check = this.interlocks.get(outputId);
    if (!check) return true;  // No interlock defined

    return check();  // Returns true if safe to operate
  }

  async writeWithInterlock(outputId: string, value: number): Promise<boolean> {
    if (!this.checkInterlocks(outputId)) {
      console.warn(`Interlock prevents operation of ${outputId}`);
      return false;
    }

    await writeOutput(outputId, value, 10);
    return true;
  }
}

// Example: VFD interlock with filter status
const interlocks = new InterlockManager();

interlocks.addInterlock('VFD1', () => {
  const filterClean = readInput('Filter_Status') === 0;
  const smokeAlarm = readInput('Smoke_Alarm') === 0;
  return filterClean && !smokeAlarm;
});
```

Next Steps

- REST API Reference
- Modbus Register Mapping

- PID Loop Tuning